Trabalho Prático Nº2 – *Packet Sniffer*
Duração: 12h

Trabalho Prático — Desenvolvimento de um Packet Sniffer e Analisador de tráfego

1. Objetivo

Neste projeto, os alunos irão desenvolver um *packet sniffer* utilizando a linguagem de programação com a qual se sintam mais confortáveis (por exemplo: Python, C/C++, Go, Java, Rust, entre outras). A aplicação deverá ser capaz de capturar pacotes de rede de forma semelhante a ferramentas como o Wireshark, tendo como principal objetivo analisar, classificar e caracterizar o tráfego que flui numa rede.

O *sniffer* irá identificar e distinguir os diferentes protocolos de comunicação abordados nas aulas teóricas, analisando os campos relevantes dos cabeçalhos e o comportamento típico de cada protocolo. Para além da simples identificação, o sistema será também capaz de classificar o tráfego por tipo e associá-lo aos respetivos protocolos.

O desenvolvimento do *sniffer* é realizado em duas fases distintas:

1. Rede emulada no CORE — Será utilizado um ambiente controlado e reprodutível para validar as funcionalidades implementadas, permitindo observar e analisar o comportamento dos protocolos estudados em cenários bem definidos;
2. Interface real do PC (Wi-Fi/Ethernet) — Nesta fase, o *sniffer* deverá ser executado sobre uma interface de rede real, possibilitando a análise de tráfego real e a observação de protocolos e padrões de comunicação que não são facilmente reproduzíveis no ambiente emulado do CORE.

Por fim, para cada protocolo identificado, os alunos deverão reconhecer e descrever os passos característicos da interação, como por exemplo fases de estabelecimento de ligação, troca de mensagens de controlo, transferência de dados e terminação da comunicação. Esta análise deverá permitir relacionar o tráfego capturado com o funcionamento interno de cada protocolo, reforçando a compreensão dos seus mecanismos fundamentais.

2. Requisitos gerais

O *sniffer* desenvolvido deve:

- Capturar pacotes numa interface de rede (no CORE e no PC).
- Mostrar (e/ou guardar) informação mínima por pacote:
  - *timestamp* (data/hora ou tempo relativo),
  - interface,
  - protocolo identificado (ex.: ARP, IPv4, ICMP, etc),
  - endereços relevantes (MAC e/ou IP),
  - tamanho do pacote,
  - resumo do conteúdo (ex.: “ARP request”, “ICMP echo request”, “DHCP Discover”, etc.);
- Permitir filtros, como por exemplo:
  - por protocolo,
  - por IP,
  - por MAC,
  - ou por expressão estilo BPF (caso usem uma biblioteca que suporte);
- Ter um modo de execução “live” (imprime na consola) e um modo “log” (guarda em ficheiro .txt/.csv/.json), podendo estar os dois ativos simultaneamente;
- Permitir identificar as trocas características de cada protocolo estudado nas aulas teóricas, associando um diagrama do funcionamento do protocolo à captura observada.

3. Parte A — Instanciação e captura na rede do emulador CORE

- Criar uma topologia de rede no emulador CORE que permita demonstrar os protocolos que desejam abordar,
- Instanciar o *sniffer* numa interface de um dos componentes da rede e capturar tráfego durante atividades simples, como:
  - ping entre nós,
  - transferência simples (ex.: netcat, HTTP local, ou outro serviço),
  - outras aplicações que desencadeiem as interações necessárias.

Nesta fase é expectável que o *sniffer* consiga já identificar os protocolos dos pacotes que são trocados, imprima a sua informação mais importante de forma legível, e guarde esta captura num ficheiro que pode ser posteriormente aberto e analisado.

Os protocolos a analisar deverão ser avaliados junto do docente das aulas práticas, assim como o método que será implementado para a sua identificação.

A captura correta de cada um dos protocolos escolhidos neste ambiente controlado deve ser assegurada antes de partirem para uma captura na interface real.

Devem consultar o docente das aulas práticas antes da utilização de bibliotecas. Não será aceite a utilização de bibliotecas que trivializem as funcionalidades a implementar.

Recomenda-se vivamente que para a captura dos pacotes seja utilizada uma biblioteca como o scapy (python), libpcap (C) ou o equivalente na linguagem de programação escolhida. A não utilização da mesma implica um grau de dificuldade bastante mais elevado ao nível do parsing dos pacotes.

4. Parte B — Transposição para a interface real do PC

Nesta fase, o *sniffer* deve ser executado na interface real do computador do aluno e deverá conseguir identificar e capturar todos os protocolos que já estavam presentes no ambiente de emulação CORE.

Pode, naturalmente, ser adicionada a análise e captura de protocolos que não estavam presentes na emulação do CORE, como IEEE 802.11 e DHCP, assim como outros que considerem interessantes e cuja interação consigam consistentemente desencadear.

Algumas restrições a apontar para a parte B:

- O *sniffer* deve ser usado apenas em redes autorizadas (CORE e rede do aluno);
- Não é permitido desenvolver funcionalidades de exploração (ex.: MITM, *injection*, *deauth*), nem recolher dados sensíveis;
- O foco é a inspeção passiva e interpretação de protocolos.

Nota: será necessário ter permissões a nível de Administrador/Root para a captura de pacotes numa interface real.

5. Opcionais

Além dos requisitos apontados, podem ser implementadas novas funcionalidades ao gosto do grupo. Estas podem ir desde a visualização gráfica a métricas de tráfego como débito ou RTT, e devem ser discutidas com o docente das práticas para avaliar a sua viabilidade/implementação.

6. Critérios de Avaliação

- Funcionalidade do *Sniffer* (captura, identificação e análise de protocolos) — 30%
- Filtros e parâmetros de execução — 10%
- *Logging* e persistência de dados — 5%
- Qualidade do código e arquitetura — 5%
- Opcionais – 10%
- Relatório — 25%
- Defesa e demonstração do projeto — 15%

7. Entregáveis finais

- Código-fonte do *sniffer* + instruções de execução (README):
  - dependências (bibliotecas),
  - como selecionar interface,
  - como ativar filtros e funcionalidades implementadas,
  - como correr no CORE e no PC.
- Relatório, que deverá incluir:
  - descrição da arquitetura do *sniffer*,
  - catalogação dos protocolos identificados e como é feita a sua análise. Se necessário, com recurso a excertos de código,
  - lista de filtros e parâmetros de entrada disponíveis,
  - exposição dos mecanismos de *logging* implementados,
  - descrição da topologia adotada para a demonstração no emulador CORE e como esta permite a identificação dos protocolos,
  - testes efetuados para validação do funcionamento da aplicação, devidamente explicados e fundamentados,
  - análise na interface real e obstáculos encontrados,
  - limitações da aplicação desenvolvida,
  - conclusão do trabalho efetuado.

O relatório pode seguir um formato livre ou o formato LNCS e deve ser submetido na plataforma de ensino com o nome:

RC-TP2-PL<TurnoGrupo>.pdf

(por exemplo, RC-TP2-PL11.pdf para o grupo PL11) até final do dia da aula prevista para a conclusão do trabalho.

8. Defesa do projeto

Além da entrega do código fonte e do relatório, o projeto terá também uma componente de defesa em que os alunos devem apresentar o trabalho efetuado, justificando e explicando convenientemente cada uma das funcionalidades implementadas. Esta terá lugar após a entrega do projeto, em calendário e local (pessoalmente/remoto) a definir com o docente das práticas.

Devem assegurar que todo o grupo está presente para a apresentação do projeto. A não comparência à defesa implica a não aprovação na componente prática da Unidade Curricular.

Por modo a que não haja atrasos, devem ter já todo o *setup* de demonstração preparado antes do início da defesa.